Machine Learning on the Factory Floor

Quality decisions, maintenance, and sensor feedback

The production question

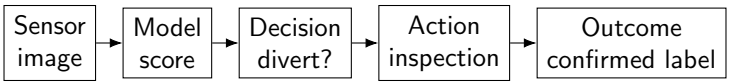
A camera inspects 1,000 robot housings per shift. A missed crack can cause an expensive return; rejecting a good housing wastes inspection time.

Today's decision

Which housings should we divert, and what evidence is needed before connecting the model to the line?

By the end: turn scores into decisions, test future performance, allocate maintenance effort, and explain a sensor feedback failure.

One chain from measurement to outcome



A good score alone does not specify the action. We also need the cost of errors, line capacity, and the time at which each measurement becomes available. All numbers today are teaching examples, not reported factory performance.

Define the label before fitting a model

Let $y = 1$ mean a crack confirmed by independent inspection. Each row needs a part ID, production time, image/features, and eventual label.

- Predict *before* diversion; exclude inspection results and rework status from input features.
- Keep images of the same part/batch together across data splits.
- Define how delayed labels are joined back to the original prediction.

If only diverted parts get inspected, accepted parts have missing outcomes. This is selective labeling, not evidence that all accepted parts were good.

A baseline exposes misleading accuracy

Only 20 of the 1,000 parts are defective. Accepting everything gives $980/1000 = 98\%$ accuracy, but misses every defect. A candidate produces:

	Defective	Good
Divert	18	40
Accept	2	940

Accuracy is 95.8%, recall $18/20 = 90\%$, precision $18/58 \approx 31\%$. Check: Would you reject the model because its accuracy is lower?

Answer with the cost of the action

Teaching cost model: diverting a good part costs 10 PLN; accepting a defective part costs 200 PLN. Correct decisions have zero *incremental* cost.

$$C = 10\,\mathrm{FP} + 200\,\mathrm{FN}.$$

Policy	Cost per 1,000 parts (PLN)
Accept all	$20(200) = 4,000$
Candidate	$40(10) + 2(200) = 800$
Divert all	$980(10) = 9,800$

The candidate is preferable under these costs despite lower accuracy. A different cost or a diversion-capacity constraint can change that conclusion.

Derive the probability threshold

Suppose $p = P(y = 1 \mid x)$ is calibrated for the population at decision time.

$$E[C \mid \text{accept}, p] = 200p,$$
$$E[C \mid \text{divert}, p] = 10(1 - p).$$

Divert when $10(1 - p) < 200p$, hence

$$\boxed{p > \frac{10}{10 + 200} \approx 0.0476.}$$

The usual 0.5 threshold silently assumes equal error costs here. An inspection charge paid for *every* diverted item would give a different equation.

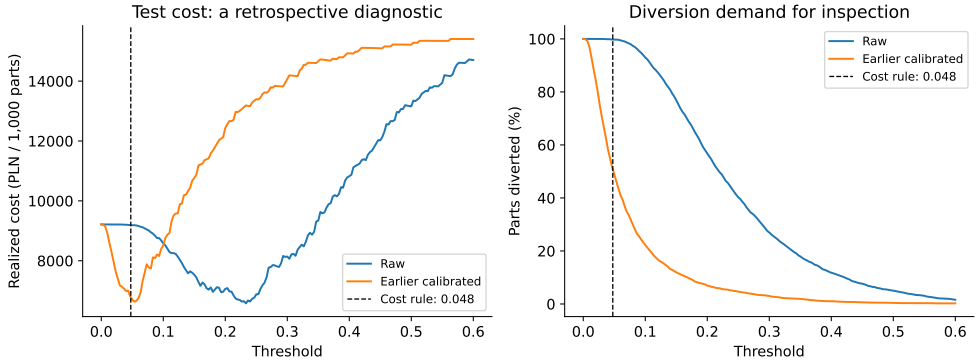
Try four parts before using Python

p	Accept cost	Divert cost	Action
0.02	4	9.8	accept
0.06	12	9.4	divert
0.15	30	8.5	divert
0.55	110	4.5	divert

Costs are expected PLN per part, not certain outcomes. Check: If the missed-defect cost doubles, should the threshold rise or fall?

Answer: fall to $10/(10+400) \approx 0.0244$; avoiding missed defects becomes more valuable.

A probability threshold also creates an inspection queue



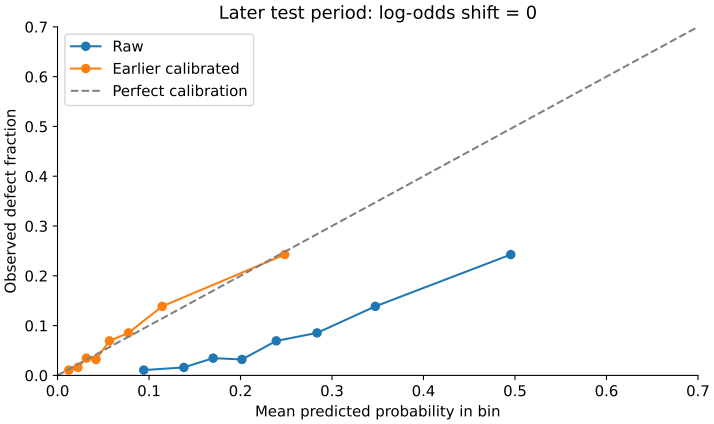
Same 3,000-part synthetic test sample in both panels. The dashed line is derived from costs, not fitted to the test curve. Lower thresholds demand more inspection capacity.

A score is not automatically a probability

A model can rank parts correctly while overstating risk: replacing p by $\sqrt{p}$ preserves ranking but changes decisions made using the same numerical threshold. Calibration asks: among parts assigned risk near 0.10, is the defect fraction near 10%?

- Plot mean predicted probability against observed defect fraction in bins.
- Show bin counts; rare defects create uncertain estimates.
- Compare Brier score $n^{-1} \sum_i (p_i - y_i)^2$ as a probability score, not a pure calibration measure.

Same ranking, different probability calibration



Fixed synthetic model; calibration uses an earlier validation period. Each point summarizes 375 later test parts. Ranking survives score inflation; the diagonal probability interpretation does not.

Separate fitting, calibration, selection, and testing

Period	Allowed use
Early data	Fit feature transforms and prediction model
Later validation	Fit calibration; compare candidate policies
Latest untouched period	Estimate future cost and error rates

Do not tune the threshold repeatedly on the final test period. With repeated tuning, use another held-out selection period or time-based resampling. A random row split can mix adjacent images, the same batch, and future operating conditions into training. Fit scalers and imputers using training data only.

Test the question the factory will actually ask

Deployment on the same machine next month needs a chronological test. Deployment on a new machine needs a machine-held-out test, often also ordered in time. Report confusion counts and cost at the selected threshold, by machine, shift, and part family. A confidence interval or repeated time blocks makes small-sample uncertainty visible. Check: Is a low average cost enough if every miss occurs on the night shift?

Answer: no. Investigate lighting, prevalence, labels, and camera settings before rollout.

Companion experiment: scores, calibration, and drift

Open `lecture_4_quality_and_maintenance.ipynb`.

1. Predict how increasing missed-defect cost changes diversion rate.
2. Compare raw scores with scores calibrated on earlier validation data.
3. Introduce a later change in defect prevalence and inspect the reliability plot.

The generator's true probabilities are available only for teaching. A real factory must estimate performance from delayed, independently checked outcomes.

A practical model can be simple

For a few tabular process measurements, start with a transparent baseline such as logistic regression:

$$p(x) = \frac{1}{1 + \exp[-(\beta_0 + \beta^T x)]}.$$

For images, compare a learned vision model with a basic image-processing rule and the current inspection process. Deep learning learns useful representations, but still needs choices about cropping, illumination, labels, augmentation, and operating thresholds. More complexity does not guarantee better future cost.

From a single decision to line capacity

Suppose inspection can handle only 30 parts per shift, but 58 cross the cost threshold. For part i, the expected benefit of diversion is

$$b_i = 200p_i - 10(1 - p_i).$$

With equal inspection time, choose at most 30 parts with the largest *positive* benefits. If inspection times differ, selection becomes a capacity-allocation problem. Capacity changes the policy; it does not make an uncalibrated score a probability. Log the policy version along with the model version.

Maintenance needs a horizon and an intervention

Now consider three robot gearboxes. Predict failure *during the next seven days*, using measurements available at the start of that horizon. A failure label is not the same as an anomaly label. A temperature spike may be a healthy high-load cycle. Specify what maintenance does: in this teaching model it costs 600 PLN and prevents all failures in the seven-day horizon. Waiting costs 10,000 PLN if a failure occurs. These assumptions must be checked against maintenance records.

Maintenance: a second decision rule

$$E[C \mid \text{maintain}] = 600, \qquad E[C \mid \text{wait}] = 10,000p.$$

Maintain if $p > 0.06$ when there is no crew constraint.

Machine	p	Wait cost	Saving from maintenance
A	0.02	200	−400
B	0.08	800	200
C	0.15	1,500	900

One available crew slot: service C. Two slots: service B and C. Ranking only by risk works here because intervention costs and avoided losses are identical.

Why maintenance labels are difficult

If we replace a bearing after an alarm, we do not observe when that bearing would otherwise have failed. Its future lifetime is censored by our intervention.

- Keep timestamps for operating exposure, alarms, repairs, and failures.
- Define a positive horizon label only when follow-up is sufficient.
- Do not label every preventive replacement as an observed failure.
- For remaining-life models, consider censoring and usage intensity explicitly.

Evaluate the maintenance policy over repeated horizons without counting the same impending failure as many independent successes.

When maintenance does not eliminate every failure

Suppose maintenance leaves residual failure probability q, while doing nothing has failure probability p.

$$E[C \mid \text{maintain}] = 600 + 10,000q.$$

The intervention is worthwhile if

$$10,000(p - q) > 600.$$

Check: For machine B, $p = 0.08$ and $q = 0.03$. Maintain?
Answer: no under this model: avoided loss is 500 PLN, below the 600 PLN intervention cost. Predictive accuracy alone cannot establish intervention benefit.

A sensor can make the controller confidently wrong

The true process is x_t, the sensor reads z_t, and the actuator command is u_t. The companion simulates

$$z_t = x_t + b_t + \epsilon_t,$$
$$u_t = \mathrm{clip}_{[0,2]}[r + K(r - z_t)],$$
$$x_{t+1} = x_t + 0.15(u_t - x_t) + w_t.$$

Here $r = 1$ is the target, $K = 2$ is the feedback gain, and ϵ_t, w_t are small measurement/process noise terms. At step 70, sensor bias becomes $b_t = 0.3$. Predict: will the true process rise or fall after the bias appears?

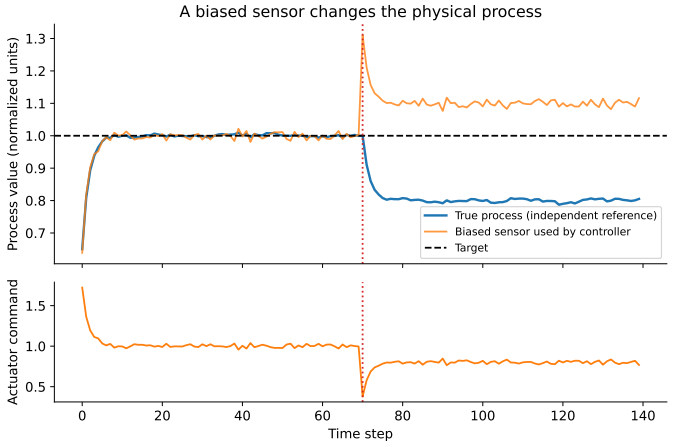
Derive what the controller will settle to

Ignore noise and assume the actuator is not saturated. At equilibrium $x_{t+1} = x_t = x^*$, so the process equation requires $u^* = x^*$.

$$x^* = r + K(r - x^* - b),$$
$$(1 + K)x^* = (1 + K)r - Kb,$$
$$\boxed{x^* = r - \frac{K}{1+K}b}, \qquad z^* = r + \frac{b}{1+K}.$$

For $r = 1$, $K = 2$, $b = 0.3$: the true value approaches 0.8 while the sensor reads 1.1. Higher gain moves the displayed value closer to target but the true value farther below it.

A biased measurement changes the physical process



Bias begins at step 70. Target $= 1$, gain $= 2$, bias $= 0.3$: true equilibrium $1 - 2(0.3)/3 = 0.8$.
A better-looking display does not establish a correct physical state.

Separate predictions from protected control

Use a staged deployment:

1. Replay historical timestamped inputs and check latency and data joins.
2. Run in shadow mode; log proposed decisions and actual outcomes.
3. Pilot with operator review and a documented fallback for missing or stale inputs.
4. Expand only after measured cost, capacity, and reliability meet agreed criteria.

A learned quality model can recommend diversion. Protective stops and physical limits need their own engineered validation; a model score alone is not a safety guarantee.

What to monitor after deployment

Signal	Interpretation and next action
Missing/stale sensor	Check acquisition; use the validated fallback
Score or feature shift	Investigate operating changes; shift alone does not prove errors
Confirmed defect rate, calibration, cost	Compare after labels arrive; examine subgroups
Diversion count and queue	Check capacity and downstream delays

Retraining on automatically selected labels can reinforce old blind spots. Retain an independent audit sample of accepted parts.

Mini design review: answer before the next slide

A camera upgrade doubles all defect scores but leaves part ranking unchanged. The team keeps the same threshold and celebrates higher recall.

1. What else must be measured?
2. Should the old probability threshold still be trusted?
3. What evidence would justify deployment?

Design review: a defensible answer

Measure false diversions, missed defects, inspection queue, and total incremental cost on independently labeled later data. The old numeric threshold is justified only if scores still represent calibrated probabilities under the same loss model. Preserving ranking does not preserve calibration. Recheck the measurement pipeline and calibrate using separate validation data. Compare with the previous policy in a controlled pilot. More alarms alone do not establish improvement.

Prepare for the laboratory

The upstream laboratories build larger decision models; this companion isolates measurement, uncertainty, and intervention assumptions.

- Export probabilities and horizon definitions, not unexplained scores.
- Give the optimization layer action costs, capacities, and feasible actions.
- Recompute outcomes when actions change; a prediction is not proof of causation.

Before leaving: explain why a 95.8% accurate classifier can beat a 98% baseline, and why a well-ranked score can still produce an expensive policy.

Sources and reproducibility

- scikit-learn: Probability calibration: reliability diagrams, separate calibration data, probability scores.
- scikit-learn: time series validation: evaluate future observations using earlier training periods.
- Automation and robotics labs: complementary applied exercises; companion data here are synthetic.

Notebook uses NumPy, SciPy, pandas, Matplotlib, and optional ipywidgets. Fixed random seeds make the displayed experiment repeatable; controls also have direct function calls.